

Midterm L5101



Please check Quercus for your curved grade.

My score

23% (8.5/37)

Q1

3

Correct **1**

Correct **1**

Correct **1**

Incorrect **0**



EA8CFC1C-EA23-4285-A413
midTerm-15101-cf809
#3 Page 2 of 12

Incorrect 0

MIDTERM TEST
CSC 209 H1S
Duration: 50 min

Q2a

0

Question 1. Answer the following by circling either the **T** (for true) or **F** (for false) that precedes each question. Any ambiguous circling will be simply considered wrong, please use caution. [5 MARKS]

Part (a) T | **F** You often need to store strings on the heap using `strdup` or using `malloc` because you cannot null-terminate strings on the stack. [1 MARK]

Part (b) T | **F** The read-only section of an executable is named "read-only", because it is only there for when we read the executable using commands like `hexdump` or `objdump`, like we did in class. [1 MARK]

Part (c) **T** | F If you wanted to give sufficient permission for any system user to execute your compiled code, you could use either `chmod 777` or `chmod a+x`. [1 MARK]

Part (d) T | **F** If you were so motivated, you could use a pointer (e.g. `char *x`) to store the value of a triple pointer (e.g. `char ***y`). In the sense, that you could store the same address with the single pointer (i.e. it is possible for `x` to have the same, non-trivial value, as `y`). [1 MARK]

Part (e) **T** | F Supposing I have an array: `int a[100]`; I can use the variable `a` *exactly* like a pointer to the first element of the array. [1 MARK]

Question 2. Answer these short answer questions in a minimal amount of space. Any answer requiring 3 or less *statements* will be awarded full marks. Each additional *statement* will result in a 1 mark deduction, the minimum score is 0. Note that brackets on an empty line are *not* considered statements, and you will not be deducted for these. [7 MARKS]

Part (a) Copy a string from a variable `char *s`, into the destination pointed to by `char *d`. Variable `d` is guaranteed to be of sufficient size and `s` is null-terminated. You may modify either pointer, but you cannot use any functions from `string.h`, e.g. `strcpy` or `strlen`. [3 MARKS]

f

MIDTERM TEST
CSC 209 H1S
Duration: 50 min

936BF122-1DBE-4146-851B-E5BA44C6ECE1
midterm-15101-cf809
#3 Page 3 of 12



Q2b

2

Part (b) Add statements above the `printf` statement, so that "12" is ultimately printed. [2 MARKS]

each num: 4 bytes
 $3 \times 4 = 12 \text{ bytes}$

```
int s[] = {2, 4, 5};  
printf("%d\n", sizeof(s));
```

Q2c

2

Part (c) Create an array of 100 integers on the heap, set the first 50 elements to 100, and the second 50 to -100. No error checking is required [2 MARKS]

```
int *arr = malloc(sizeof(int) * 100);
```

```
for (int i = 0; i < 100; i++)
```

```
{  
    arr[i] = 100;  
    arr[99-i] = -100;  
}
```

```
}
```



308695F1-BACF-4016-BC98-3750612F72B2
 midterm-15101-cf809
 #3 Page 4 of 12

MIDTERM TEST
 CSC 209 H1S
 Duration: 50 min

Question 3. Consider the following program which uses selection sort to order an array of students by their score. [12 MARKS]

```

1  typedef struct student {
2      const char * name;
3      int id;
4      float score;
5  } Student;
6
7  /* returns negative value if a < b, positive if a > b, and 0 when a == b. */
8  int compare_score(const Student * a, const Student * b) {
9      return a->score - b->score;
10 }
11
12 void swap_student(Student a, Student b) {
13     Student temp = a;
14     a = b;
15     b = temp;
16 }
17
18 void selection_sort(int size,
19                     Student * arr[size],
20                     int (* compare)(const Student *, const Student *),
21                     void (* swap)(Student, Student)) {
22     int min_idx;
23     for (int i = 0; i < size-1; i++) {
24         min_idx = i;
25         for (int j = i+1; j < size; j++) {
26             if (compare(arr[j], arr[min_idx]) < 0)
27                 min_idx = j;
28         }
29         if (i != min_idx)
30             swap_student(*arr[i], *arr[min_idx]);
31     }
32 }
33
34 int main() {
35     Student jean = {"Jean Valjean", 24601, 87.},
36               max = {"Max Short", 32767, 66.},
37               yi = {"Yi Wan", 10000, 72.};
38     Student * arr[] = {&jean, &max, &yi};
39
40     const int size = sizeof(arr)/sizeof(Student *);
41     selection_sort(size, arr, compare_score, swap_student);
42
43     for (int i = 0; i < size; i++)
44         printf("%d. %d: %.1f%%\n", i+1, arr[i]->id, arr[i]->score);
45
46     return 0;
47 }

```

MIDTERM TEST

CSC 209 H1S

Duration: 50 min

BB58416B-641A-4CE5-BB34-BCF6A7A773AF

midterm-15101-cf809

#3 Page 5 of 12



Q3a

0.5

Section	Address	Value	Label
text	0x100	function code	main
	0x200	function code	selection_sort
	0x300	function code	swap_student
	0x380	function code	compare_score
rodata	0x400	Jean	
	0x404	Val	
	0x408	jean	
	0x40c	\0	
	0x410	Max	
	0x414	Shor	
	0x418	t\0	
	0x41c	Yi W	
	0x420	an\0	
	0x424	%d.	
	0x428	%d:	
	0x42c	%.1f	
	0x430	%%\n\0	
	0x434		
	0x438		
	0x43c		
stack <i>SelectionSort</i>	0x898		Size
	0x89c		
	0x900		
	0x904		
	0x908		
	0x90c		
	0x910		

stack table continues on next page...



0E848B9C-8579-42CF-ADAC-BE8A65D937FF
midterm-15101-cf809
#3 Page 6 of 12

MIDTERM TEST
CSC 209 H1S
Duration: 50 min

Section	Address	Value	Label
stack (continue)	0x914		
	0x918		
	0x91c		
	0x920		
	0x924		
	0x928		
	0x92c		
	0x930		
	0x934		
	0x938		
	0x93c		
	0x940		
	0x944		
	0x948		
	0x94c		
	0x950		
	0x954		
	0x958		
	0x95c		
	0x960		
	0x964		
	0x968		
	0x96c		
	0x970		
	0x974		
	0x978		
	0x97c		
	0x980		
	0x984		
	0x988		
	0x98c		
	0x990		
	0x994		
	0x998		
	0x99c		

MIDTERM TEST

CSC 209 H1S

Duration: 50 min

FFA16CC3-DE67-4F70-B874-E68D402D5469

midterm-15101-cf809

#3 Page 7 of 12



Part (a) Fill in the memory model for this code and stop just before `swap_student` returns for the first time, i.e., `swap_student` has run once. The final values in each relevant memory address are sufficient, but you may show your work by crossing out old values that changed. You can assume you are working on a 64-bit machine. Note: you need to create your own stack frames for each function call, starting with `main`. [7 MARKS]

Q3b

0

Part (b) There is a currently a bug with the program. State the bug and rewrite the faulty code in the space below. Clearly indicate where you are adding, removing, or replacing code, by specifying the relevant line number(s) [5 MARKS]

unanswered 0



6A855FA2-6A9E-4AC1-BBF5-7A3D502260EB

midterm-15101-cf509

#3 Page 8 of 12

MIDTERM TEST

CSC 209 H1S

Duration: 50 min

Question 4. Consider the following **Makefile** and answer the questions below. The same rules from Q1 apply to the true and false question. The initial contents of the project directory are the following files: **Makefile** (below), **parse.c**, **calculate.c** and **tally.c**. [7 MARKS]

```
CC := gcc
CFLAGS = -std=gnu99 -Wall

OBJS = parse.o calculate.o tally.o
TARGET := tally
EXTRA_FILE = extra-file.txt

① $(TARGET) : $(OBJS)
    $(CC) $(CFLAGS) -o $(TARGET) $^

all : ${TARGET} extra

extra :
    ./$(TARGET) 576 > $(EXTRA_FILE)

%.o: %.c
    $(CC) $(CFLAGS) -c $<
```

Q4a

1

Part (a) T | ☒ F The executable **tally** can be debugged with **gdb** if you build with the command **make all**. [1 MARK]

Q4b

0

Part (b) If you typed **make** at the command line, which *rule* is evaluated first? [1 MARK]

① \$(target); \$(OBJS) \$(CC)

Q4c

0

Part (c) Write an additional rule called **clean** to clean this project so that any files created by invoking **make** with this **Makefile** (as it is above, before any additions you've made) are removed. [2 MARKS]

Q4d

0

MIDTERM TEST

CSC 209 H1S

Duration: 50 min

A953CF84-727B-41F9-8602-244BEEDA560D

midterm-15101-cf809

#3 Page 9 of 12



Part (d) Add an additional rule called `test`, which creates a new directory in the project called `test-dir` and moves `extra-file.txt` into this directory. [2 MARKS]

Q4e

0

Part (e) Add an additional `.PHONY` target with appropriate prerequisites for the `Makefile` (as it is above, before any additions you've made). [1 MARK]



2D6CF756-CCB3-48C7-9445-10E4E246B3DD

midterm-15101-cf809

#3 Page 10 of 12

MIDTERM TEST

CSC 209 H1S

Duration: 50 min

Question 5. You have been asked to write a function that will refresh (*re/load* data from disk) into *both* of the arrays of a `struct dataset` (defined below, the beginning of the arrays are members `x` and `y`). A second `struct` is used as a "header" and it is stored at the byte offset indicated by `HEADER_OFFSET` within any compatible file. The header stores the byte offsets in the file at which the arrays pointed to by the members of a `struct dataset` can be read. On the next page, complete the function `refresh_arrays` to load the arrays from file between the specified indices *inclusively*. You can assume all system calls work perfectly, the file exists and is compatible, and indices are within range – no error checking is required. [6 MARKS]

```
#define HEADER_OFFSET 0x42

struct dataset{
    char name[256];
    unsigned long x_len;
    unsigned long y_len;
    float *x;
    float *y;
};

struct header{
    unsigned long x_offset;
    unsigned long y_offset;
};
```

Q5

0

MIDTERM TEST
CSC209 H1S
Duration: 50 min

017ADF25-3503-4038-A524-FD9E86EB5D8D
midterm-15101-cf809
#3 Page 11 of 12



```
void refresh_dataset(char *filename, struct dataset *data, int start_index,  
                    int end_index){
```

```
}
```



9F893576-DF87-402E-B50F-64396BCC889F

midterm-15101-cf809

#3 Page 12 of 12

MIDTERM TEST

CSC209 H1S

Duration: 50 min

*Use the space on this "blank" page for scratch work, or for any solution that did not fit elsewhere.
Clearly label each such solution with the appropriate question and part number.*

Total Marks = 37

